

ESTUDIO — Carpeta util/

Material de estudio para el final del TFC Bajoneá. Basado en el código real de backend/src/main/java/com/bajonea/backend/util/ — 2 clases.

Qué es una clase utilitaria

Una **clase utilitaria** (o *helper*) es una clase que junta funciones sueltas que **no pertenecen a ninguna entidad ni a ningún service en particular**, pero que **varios necesitan**.

Se reconocen por tres características, y las dos clases del proyecto las cumplen:

Característica	Qué significa	Por qué
public final class	No se puede heredar de ella.	No tiene sentido "extender" un cajón de funciones.
Constructor privado vacío	No se puede hacer new .	No hay nada que instanciar: no guarda estado.
Todos los métodos static	Se llaman por el nombre de la clase: TextoUtils.aTitleCase(...) .	No hacen falta objetos.

```
public final class TextoUtils {
    private TextoUtils() { }    // <-- nadie puede hacer new TextoUtils()
    public static String aTitleCase(String texto) { ... }
}
```

Cuándo conviene una clase utilitaria en vez de un service: cuando la función es **pura** — le das una entrada, te devuelve una salida, no toca la base, no depende de nada externo, y siempre da lo mismo con la misma entrada. Si necesitara consultar la base o inyectar dependencias, sería un service, no un util.

1. TextoUtils — normalización de texto

Para qué sirve

Limpiar y estandarizar textos **antes de guardarlos** en la base: capitalizar nombres, arreglar URLs sin protocolo, normalizar códigos postales.

Quiénes la usan: los services que persisten nombres, apellidos, razón social, nombre de comercio, nombre de producto y calle de dirección. También la usa RedSocialRequestDTO en su setter manual.

Atributos (constantes privadas)

Constante	Qué es
LOCALE_NORMALIZACION	Locale.forLanguageTag("es-AR") — el idioma para las conversiones de mayúsculas.
FORMATO_CPA	El regex ^[A-Za-z]\d{4}[A-Za-z]{3}\$ — el formato del código postal argentino nuevo.
ESQUEMA_HTTP	El regex (?i)^https?://.* — detecta si una URL ya tiene protocolo.

Por qué el Locale importa (detalle fino que puede sumar)

Convertir a mayúsculas **no es igual en todos los idiomas**. El caso famoso es el turco: la `ı` minúscula en mayúscula da `İ` (con punto), no `I` . Si el servidor estuviera configurado en turco y usaras `toUpperCase()` sin locale, obtendrías resultados distintos.

Fijar `es-AR` explícitamente hace que **el resultado sea siempre el mismo, sin importar cómo esté configurado el servidor**. Es una buena práctica que muy poca gente aplica.

Métodos

1.1 normalizarUrlConEsquema(String url)

Qué hace: si la URL no empieza con `http://` o `https://` , le agrega `https://` adelante.

Cómo funciona:

- Si es `null` → devuelve `null` .
- Le hace `trim()` (saca espacios de los bordes).
- Si quedó vacía o ya tiene protocolo → la devuelve tal cual.
- Si no → devuelve `"https://" + url` .

Ejemplos:

Entrada	Salida
"instagram.com/mipizzeria"	"https://instagram.com/mipizzeria"
"https://facebook.com/pizza"	"https://facebook.com/pizza" (no la toca)
"HTTP://ejemplo.com"	"HTTP://ejemplo.com" (el (¿i) la reconoce igual)
" "	""
null	null

Para qué se usa: en el setter de `RedSocialRequestDTO.url` . **Nadie escribe el `https://` a mano** cuando carga su Instagram. Sin esta normalización, el link guardado sería `instagram.com/pizza` y al hacer clic el navegador lo interpretaría como una ruta relativa del propio sitio, no como un link externo. Un detalle de usabilidad que evita un bug real.

Detalle del regex: el `(¿i)` al principio significa *case-insensitive* — reconoce `HTTP://` , `Https://` y `https://` por igual.

1.2 normalizarCodigoPostal(String codigoPostal)

Qué hace: si el código postal tiene el formato **CPA** (el nuevo, alfanumérico), lo pasa a mayúsculas. Si no, lo devuelve como vino (solo con `trim`).

Cómo funciona:

- Si es `null` → `null` .
- `trim()` .
- ¿`Matchea ^[A-Za-z]\d{4}[A-Za-z]{3}$` ? → lo devuelve en **mayúsculas**.
- Si no → lo devuelve tal cual.

Los dos formatos de código postal argentino, por si preguntan:

Formato	Ejemplo	Qué es
Clásico	9420	4 dígitos. El viejo de toda la vida.
CPA	V9420ABC	Letra + 4 dígitos + 3 letras. El nuevo del Correo Argentino.

Ejemplos:

Entrada	Salida	Por qué
"v9420abc"	"V9420ABC"	Es CPA → a mayúsculas.
"V9420ABC"	"V9420ABC"	Ya está bien.
"9420"	"9420"	Es clásico, no aplica.
" 9420 "	"9420"	Solo el trim.

Por qué normalizar solo el CPA: el CPA es alfanumérico, y por convención va en mayúsculas. El clásico son solo dígitos, así que pasarlo a mayúsculas no haría nada. Aplicar la regla solo donde tiene sentido evita transformaciones inútiles.

1.3 aTitleCase(String texto) — el método más importante

Qué hace: convierte un texto a **Title Case** — primera letra de cada palabra en mayúscula, el resto en minúscula.

Cómo funciona, paso a paso:

- Si es `null` → `null` .
- Pasa **todo a minúsculas** primero (con el locale `es-AR`).
- Recorre carácter por carácter con un `StringBuilder` , llevando una bandera `inicioDePalabra` .
- Si está al inicio de una palabra y el carácter es una letra → la pone en **mayúscula**.
- Si no → la deja como está, y actualiza la bandera preguntando si ese carácter es un delimitador.

Los **delimitadores de palabra** (método `esDelimitadorDePalabra`):

Carácter	Ejemplo de por qué
Espacio	"juan carlos" → "Juan Carlos"
Guion -	"garcia-lopez" → "Garcia-Lopez"
Apóstrofo '	"o'brien" → "O'Brien"
Barra /	"pizza/empanadas" → "Pizza/Empanadas"

Ejemplos completos:

Entrada	Salida
"JUAN CARLOS PEREZ"	"Juan Carlos Perez"
"maría josé"	"María José"
"garcia-lopez"	"García-Lopez"
"o'brien"	"O'Brien"
"PIZZAS DEL SUR"	"Pizzas Del Sur"

Por qué se hizo a mano y no con una librería

Java **no trae** una función de Title Case en su biblioteca estándar. Existe `WordUtils.capitalizeFully()` de Apache Commons, pero traer una dependencia entera por una sola función no vale la pena.

Y hay un motivo más importante: este método es una **portación exacta** de la función `aTitleCase` que existe en `frontend/js/validators.js`.

El detalle clave: backend y frontend capitalizan igual

Ese es el punto que vale la pena contar en la mesa.

Por qué importa: el frontend le muestra al usuario cómo va a quedar su nombre mientras lo escribe. Si el backend capitalizara **distinto**, el usuario vería una cosa en el formulario y otra después de guardar. Peor: en los tests E2E de Playwright, una comparación entre lo que se escribió y lo que se guardó fallaría por una diferencia de mayúsculas.

De hecho eso pasó: cuando se agregó la normalización a Title Case en el backend, **rompió 7 tests de Playwright** que esperaban el texto tal cual se había escrito. Es una de las 7 regresiones de "drift" que se detectaron y corrigieron al re-verificar la suite E2E.

Cómo lo contás: *"Tengo la misma lógica de capitalización en Java y en JavaScript, portada a mano, para que backend y frontend muestren exactamente lo mismo. Si divergieran, el usuario vería un texto en el formulario y otro después de guardar."*

El riesgo de esta decisión, dicho de frente: son **dos copias de la misma lógica**, y si mañana cambiás una tenés que acordarte de cambiar la otra. En un proyecto más grande se resolvería con una sola fuente compartida, pero para dos funciones chicas en dos lenguajes distintos, duplicar es más simple que armar infraestructura para compartir.

2. ComercioValidaciones — una regla de negocio compartida

Para qué sirve

Guarda una sola regla de negocio de comercio que se usa en dos lugares distintos.

Métodos

2.1 validarModalidadesEntrega(boolean aceptaDelivery, boolean aceptaRetiro)

```
public static void validarModalidadesEntrega(boolean aceptaDelivery, boolean aceptaRetiro) {
    if (!aceptaDelivery && !aceptaRetiro) {
        throw new ValidacionException(
            "El comercio debe ofrecer al menos una modalidad de entrega (delivery o retiro)");
    }
}
```

Qué valida: que un comercio ofrezca **al menos una** de las dos modalidades. Puede ofrecer delivery, puede ofrecer retiro, puede ofrecer las dos. **Lo que no puede es no ofrecer ninguna** — sería un comercio al que nadie puede pedirle nada.

Qué hace si falla: lanza `ValidacionException`, que el `GlobalExceptionHandler` traduce a **400 Bad Request**.

Las dos preguntas que este archivo responde

"¿Por qué no está como anotación en el DTO?"

Porque Bean Validation **valida un campo a la vez**. Acá la regla depende de **la combinación de dos campos**: ni `aceptaDelivery` ni `aceptaRetiro` son inválidos por sí solos, lo inválido es que los dos sean `false` **al mismo tiempo**.

Se podría haber hecho una anotación custom a nivel de clase (como `@DireccionExclusionMutua`), pero para un solo caso simple no valía la pena.

"¿Por qué no está adentro de un service?"

Porque la usan dos. La regla vale tanto al **registrar** un comercio (`RegistroService`) como al **editar** su perfil (`ComercioService.editarPerfil`).

Originalmente estaba escrita dentro de `RegistroService`. Cuando se implementó la edición de perfil, había dos opciones:

1. Copiar y pegar el `if` → si mañana cambia la regla, hay que acordarse de cambiar los dos, y tarde o temprano se desincronizan.
2. Extraerla a un lugar común → una sola definición, ambos la llaman.

Se eligió la segunda. Es un ejemplo concreto de **DRY** (*Don't Repeat Yourself*), y es una buena respuesta si te preguntan por qué existe esta clase.

Comparación: dónde va cada tipo de validación

Esta tabla es muy útil para la mesa, porque la pregunta "¿dónde ponés cada validación?" es de las que más se hacen:

Tipo de validación	Dónde va	Ejemplo del proyecto
Formato de un campo	Anotación en el DTO	@Email en email
Formato complejo de un campo	Anotación custom	@ValidarCuit (dígito verificador)
Combinación de dos campos, un solo lugar	Directo en el service	direccionId obligatorio si es domicilio
Combinación de dos campos, varios lugares	Clase utilitaria	ComercioValidaciones
Regla que consulta la base	Service	Que el email no esté ya registrado
Regla de unicidad , garantía final	UNIQUE de la base	El email en la tabla usuario

Preguntas típicas de mesa

"¿Qué hay en `util`?" Dos clases: `TextUtils` , con funciones de normalización de texto (Title Case, URLs, código postal), y `ComercioValidaciones` , con una regla de negocio compartida por dos services.

"¿Por qué son clases estáticas con constructor privado?" Porque no guardan estado. No tiene sentido instanciarlas ni heredar de ellas: son un cajón de funciones puras.

"¿Por qué duplicaste la función de Title Case en Java y JavaScript?" Para que backend y frontend capitalicen exactamente igual. Si divergieran, el usuario vería un texto en el formulario y otro distinto después de guardar. De hecho, al agregarla en el backend rompió 7 tests de Playwright que esperaban el texto sin normalizar.

"¿Por qué `ComercioValidaciones` no es una anotación de Bean Validation?" Porque depende de la combinación de dos campos, y Bean Validation valida un campo a la vez. Y no está adentro de un service porque la necesitan dos: el de registro y el de edición de perfil.

"¿Por qué fijás el Locale al convertir mayúsculas?" Para que el resultado no dependa de la configuración regional del servidor. En algunos idiomas (el turco es el caso clásico) la conversión de la `i` da un carácter distinto.

Índice de archivos cubiertos en este documento

Archivo	Métodos	Qué hace
TextUtils.java	<code>normalizarUrlConEsquema</code> , <code>normalizarCodigoPostal</code> , <code>aTitleCase</code> , <code>esDelimitadorDePalabra</code> (privado)	Normalización de texto compartida por varios services. Su <code>aTitleCase</code> es idéntico al del frontend.
ComercioValidaciones.java	<code>validarModalidadesEntrega</code>	Exige que un comercio ofrezca al menos delivery o retiro. Usada por <code>RegistroService</code> y <code>ComercioService</code> .